

Proyecto #3: Redes programables

Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

CI-0121 Redes de comunicación de datos

I Semestre – 2026

Estudiantes:

Anferhny Berrocal Rojas (C11056)

Adrián Arias Vargas (C30749)

[Link al repo](#)

Introducción

En este proyecto se desarrolló una caché DNS en el plano de datos utilizando P4, BMv2 y Mininet. La motivación principal fue explorar cómo un switch programable puede realizar más que solo reenviar paquetes, permitiendo responder consultas frecuentes directamente desde la red y reduciendo la dependencia del servidor DNS.

El tema es importante porque DNS es uno de los servicios más utilizados en redes, y muchas consultas se repiten constantemente. En una red tradicional, cada solicitud debe llegar hasta el servidor DNS para obtener una respuesta, aunque el dominio ya haya sido consultado antes. Con una caché cercana a los clientes, el tiempo de respuesta puede reducirse y también se puede disminuir la carga sobre el servidor.

Para probar esta idea, se construyó una topología con cuatro clientes, un switch P4 y un servidor DNS simulado. El switch identifica consultas DNS tipo A, revisa si el dominio está en caché y responde directamente cuando hay una coincidencia. Cuando no existe una entrada guardada, la consulta se reenvía al servidor. Además, se utilizó un controlador en Python para cargar entradas iniciales, actualizar el caché y consultar contadores.

Con esta implementación se busca demostrar, de forma práctica, cómo las redes programables pueden mover parte del procesamiento hacia el plano de datos. El proyecto no pretende reemplazar un sistema DNS completo, sino mostrar un caso concreto donde P4 permite mejorar la respuesta de consultas repetidas y comparar el comportamiento entre respuestas desde caché y respuestas reenviadas al servidor.

Estado del arte

Antes de desarrollar este proyecto, ya existían varias líneas de trabajo relacionadas con redes programables, procesamiento en el plano de datos y almacenamiento de información dentro de switches. Estas ideas son importantes porque muestran que el proyecto no surge aislado, sino como una aplicación pequeña de conceptos que ya se han usado en investigaciones y tecnologías reales.

Uno de los puntos de partida es P4, un lenguaje diseñado para programar el comportamiento del plano de datos. A diferencia de un switch tradicional, donde el procesamiento de paquetes depende de funciones ya definidas por el fabricante, P4 permite describir encabezados, parsers, tablas de match-action y acciones propias. Esto hace posible que el switch entienda protocolos específicos y aplique lógica personalizada sobre los paquetes (P4 Language Consortium, n.d.). En el caso de este proyecto, esa idea se aplica al tráfico DNS, haciendo que el switch pueda reconocer consultas sobre UDP puerto 53 y tomar decisiones según el dominio consultado.

También existen recursos prácticos como los tutoriales oficiales de p4lang, donde se muestran ejemplos de forwarding básico, parsers personalizados, tablas, contadores y comunicación entre el plano de control y el plano de datos. Estos ejemplos sirven como base para entender cómo se estructura un programa P4 y cómo se prueba en BMv2 y Mininet (P4 Project, n.d.). Nuestro proyecto sigue esa misma línea, pero cambia el objetivo: en lugar de limitarse al reenvío IPv4, agrega una función de caché para responder ciertas consultas desde el propio switch.

En redes modernas también se han usado mecanismos programables para encapsulación y forwarding avanzado. Por ejemplo, VXLAN permite extender redes de capa 2 sobre una infraestructura de capa 3 mediante encapsulación UDP, lo que muestra cómo se pueden agregar nuevas funciones de red sin depender únicamente de la red física tradicional (Mahalingam et al., 2014). De forma similar, SRv6 define una forma de programar comportamientos de red usando segmentos en IPv6, permitiendo que los paquetes lleven instrucciones sobre cómo deben ser

procesados en la red (Filsfils et al., 2020). Aunque VXLAN y SRv6 no son cachés DNS, sí se relacionan con la idea general de mover más lógica hacia la red y hacer que los nodos intermedios participen activamente en el procesamiento.

También hay trabajos enfocados en seguridad y mitigación en tiempo real dentro del plano de datos. Hula et al. (2020) plantean el uso de planos de datos programables para detectar y mitigar ataques DDoS de forma rápida, aprovechando que el switch puede observar tráfico y reaccionar antes de que todo llegue a los servidores. Este tipo de propuesta muestra que el plano de datos programable puede usarse no solo para forwarding, sino también para tomar decisiones rápidas basadas en el tráfico. En nuestro caso, esa capacidad se usa con un objetivo más simple: decidir si una consulta DNS se responde desde caché o se reenvía al servidor.

En conjunto, estos trabajos muestran que existe una tendencia clara hacia redes más programables, donde los switches pueden participar en tareas que antes estaban reservadas al plano de control o a servidores externos. El proyecto de caché DNS se ubica dentro de esa tendencia, pero con un alcance reducido y medible: usar P4 para reconocer consultas DNS, guardar respuestas en registers y responder directamente cuando existe una coincidencia.

Casos de uso

Una caché DNS en el plano de datos puede ser útil en redes donde muchas consultas se repiten constantemente y donde reducir algunos milisegundos de latencia puede mejorar el rendimiento general. Aunque este proyecto se hizo en una topología pequeña con Mininet, la idea se relaciona con escenarios reales donde los dispositivos intermedios no solo reenvían paquetes, sino que también participan en el procesamiento del tráfico.

Un primer caso de uso sería una red universitaria o empresarial con muchos usuarios accediendo a los mismos servicios. En este tipo de ambiente es común

que varios clientes consulten repetidamente dominios internos, plataformas institucionales, repositorios, servicios web o herramientas de autenticación. Si esas respuestas DNS se almacenan cerca de los clientes, el switch podría responder consultas frecuentes sin enviar todo el tráfico al servidor DNS. Esto reduciría la carga del servidor y también disminuiría el tiempo de respuesta para dominios muy usados. Esta idea se relaciona con el principio de NetCache, donde los datos más consultados se almacenan directamente en la red para evitar que todas las solicitudes lleguen a los servidores finales (Jin et al., 2017).

Otro caso de uso sería en centros de datos, donde existen muchas aplicaciones distribuidas que dependen de consultas constantes para descubrir servicios. En estos ambientes, pequeñas demoras repetidas pueden afectar el rendimiento total del sistema, especialmente cuando hay muchos clientes o microservicios comunicándose entre sí. Una caché en el plano de datos podría ayudar a responder consultas conocidas de forma rápida, manteniendo el tráfico más frecuente dentro del switch programable. Esto sigue la misma tendencia de mover ciertas funciones hacia la red para mejorar latencia y reducir carga en los servidores, como se observa en propuestas de procesamiento dentro de switches programables (Jin et al., 2017; P4 Language Consortium, n.d.).

También puede aplicarse en redes de borde o edge computing. En estos escenarios, los usuarios están cerca de nodos pequeños de cómputo, pero no siempre cerca de todos los servicios centrales. Si una parte de las consultas DNS frecuentes se responde desde un switch cercano, se puede reducir el tiempo de acceso inicial a servicios repetidos. Esto es útil en ambientes como campus inteligentes, redes IoT o sistemas donde varios dispositivos consultan los mismos dominios de actualización, monitoreo o control. La ventaja de P4 en este caso es que permite definir parsers, tablas y acciones específicas para procesar tráfico de una forma más flexible que un switch tradicional (P4 Language Consortium, n.d.).

Finalmente, este tipo de solución también puede servir como base educativa o experimental. P4 permite probar ideas de red sin depender de hardware propietario, usando herramientas como BMv2 y Mininet. Los tutoriales de p4lang

muestran precisamente cómo crear laboratorios con parsers, tablas, acciones y controladores, lo cual facilita construir prototipos como este antes de pensar en una implementación sobre hardware real (P4 Project, n.d.). Por eso, aunque la caché DNS desarrollada aquí es limitada, funciona como una prueba práctica de cómo mover una función específica al plano de datos.

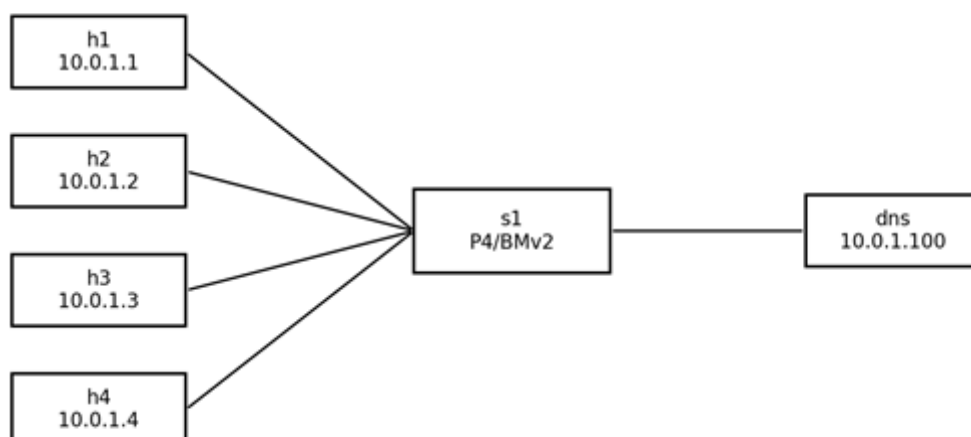
En resumen, los casos de uso más claros para una caché DNS en P4 son redes con consultas repetidas, centros de datos, edge computing, monitoreo de tráfico DNS, apoyo a seguridad y entornos educativos. En todos estos casos, el valor está en aprovechar la capacidad del switch programable para procesar tráfico directamente en la red y no depender siempre de que cada solicitud llegue hasta un servidor externo.

Diseño general del sistema

La topología se compone de un switch P4, cuatro clientes DNS y un servidor DNS simulado, como se representa en la Figura 1. Todos los hosts están en la red 10.0.1.0/24. Los clientes h1, h2, h3 y h4 generan consultas con dig hacia la IP 10.0.1.100. El host dns ejecuta un servidor DNS mínimo implementado con Scapy, el cual responde únicamente registros tipo A para dominios controlados: example.com, ucr.ac.cr y cache.test.

Figura 1.

Topología experimental: cuatro clientes, un switch P4 y un servidor DNS simulado.



El flujo esperado tiene dos caminos. En un HIT, el switch identifica una consulta DNS tipo A, calcula el índice de caché, encuentra una entrada válida y construye la respuesta sin reenviar el paquete al servidor. En un MISS, el switch no encuentra una entrada válida y reenvía la consulta al servidor DNS real o simulado usando forwarding IPv4 normal. Cuando el servidor responde, el control plane puede observar la respuesta y escribir la entrada en los registers del switch. La asignación de direcciones, MACs y puertos usados en la prueba se resume en la Tabla 1.

Figura 2.

Flujo lógico de consulta DNS: HIT desde caché P4 y MISS hacia servidor DNS.

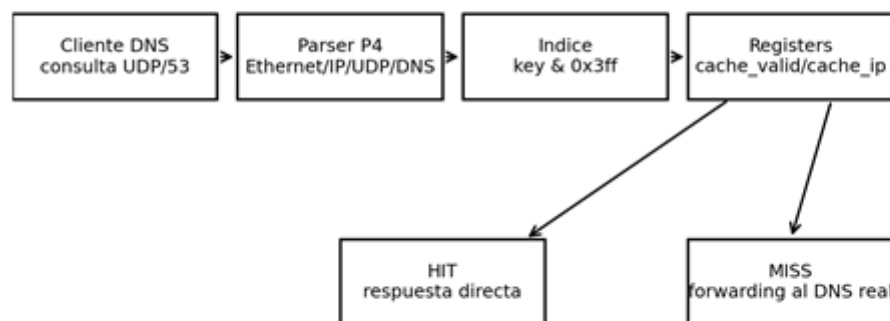


Tabla 1.

Parámetros de topología usados durante la evaluación.

Nodo	Rol	IP	MAC	Puerto en s1
h1	Cliente DNS	10.0.1.1/24	08:00:00:00:01:11	1
h2	Cliente DNS	10.0.1.2/24	08:00:00:00:01:22	2
h3	Cliente DNS	10.0.1.3/24	08:00:00:00:01:33	3
h4	Cliente DNS	10.0.1.4/24	08:00:00:00:01:44	4
dns	Servidor DNS simulado	10.0.1.100/24	08:00:00:00:01:99	5

Diseño del plano de datos en P4

El plano de datos implementa un pipeline mínimo compatible con BMv2/v1model. El parser reconoce Ethernet, IPv4, UDP y un subconjunto de DNS. La decisión inicial se toma a partir del puerto UDP destino: si el destino es 53 y el paquete corresponde a una consulta, el parser extrae el header DNS y una clave parcial del nombre de dominio. Esta clave se obtiene de los primeros cuatro bytes del nombre DNS codificado en wire format.

La caché se implementa mediante dos registers principales: `cache_valid` y `cache_ip`. El primero indica si existe una entrada válida en un índice determinado; el segundo almacena la dirección IPv4 de respuesta. El índice se calcula con una máscara de 10 bits, lo que da 1024 posiciones posibles. Esta decisión simplifica el prototipo y lo hace compatible con una función de hash ligera, aunque no elimina completamente las colisiones.

Cuando existe HIT, el switch no solo reenvía el paquete: construye una respuesta DNS. Para esto invierte las direcciones MAC, IP y puertos UDP, cambia los campos de DNS para indicar respuesta, agrega un answer tipo A y actualiza longitudes de IPv4 y UDP. En BMv2 se desactiva el checksum UDP para simplificar la validación experimental. El deparser emite los headers en el orden correcto para que dig pueda interpretar la respuesta.

Cuando no existe HIT, el paquete continúa por la tabla IPv4 LPM. Esta tabla se usa para reenviar consultas al servidor DNS y para devolver respuestas hacia los clientes. La lógica es intencionalmente simple: P4 maneja operaciones repetitivas y de baja latencia, mientras que el control plane se encarga de poblar rutas y actualizar el caché. Esta separación evita poner lógica compleja de parsing variable o política de reemplazo dentro del switch.

Control plane y población de tablas

El control plane está implementado en Python. Su primera responsabilidad es instalar las reglas de forwarding IPv4 en la tabla MyIngress.ipv4_lpm. Cada regla asocia un prefijo /32 con una MAC destino, una MAC fuente del switch y un puerto de salida. Con esto se garantiza conectividad entre clientes y servidor DNS dentro de la topología.

La segunda responsabilidad es cargar el caché inicial. En el modo normal y en auto-test, el controlador escribe tres entradas en los registers: example.com, ucr.ac.cr y cache.test. Como se puede apreciar en la Tabla 2, cada entrada tiene un índice calculado con la misma función usada por P4. Esto permite validar el escenario HIT sin levantar el servidor DNS simulado.

Tabla 2.

Entradas iniciales de caché cargadas por el controlador.

Dominio	IP cacheada	Índice	Uso
example.com	10.10.10.10	97	Prueba HIT y MISS
ucr.ac.cr	10.20.20.20	882	Prueba HIT
cache.test	10.30.30.30	355	Prueba HIT

La tercera responsabilidad es el updater dinámico. En watch-test, el controlador inicia un hilo que escucha respuestas DNS en la interfaz s1-eth5, es decir, el enlace entre el switch y el servidor DNS. Cuando observa una respuesta tipo A, extrae el nombre de dominio, la IP respondida y escribe la entrada correspondiente en los registers del switch. Esta estrategia reproduce un patrón común en redes programables: el plano de datos ejecuta la acción rápida y el control plane decide cuándo actualizar estado.

Metodología experimental

La evaluación se realizó con Mininet, BMv2 simple_switch, p4c, controlador Python, dns_server.py con Scapy y dig como cliente DNS. Se ejecutaron tres pruebas automáticas: auto-test, miss-test y watch-test. Antes de cada prueba se limpió Mininet, procesos anteriores y archivos temporales para evitar interferencia entre corridas. Estas pruebas con sus respectivos comandos y resultados se pueden apreciar en la Tabla 3.

Tabla 3.

Pruebas automáticas ejecutadas en el proyecto.

Prueba	Comando	Objetivo	Archivo de resultados
Prueba 1	sudo -E python3 topology.py --auto-test	Conectividad + HIT + latencia con caché inicial	/tmp/latency_results.txt
Prueba 2	sudo -E python3 topology.py --miss-test	MISS sin servidor y MISS con servidor Scapy	/tmp/latency_results_miss.txt
Prueba 3	sudo -E python3 topology.py --watch-test	Aprendizaje dinámico y segundo HIT desde caché	/tmp/latency_results_watch.txt

Además de las métricas de latencia, se recomienda capturar tráfico en dos puntos: en el enlace hacia el servidor DNS y en el cliente. En un HIT puro, el servidor DNS no debería recibir la consulta porque el switch responde directamente. En un MISS, sí debería observarse tráfico UDP/53 entre cliente y servidor. En watch-test, la primera consulta debe pasar al servidor y las siguientes deben responder desde caché.

La métrica principal fue el Query time reportado por dig. Para cada escenario de benchmark se realizaron 10 repeticiones por dominio o por caso. En el escenario

sin servidor, el objetivo no era medir desempeño útil, sino confirmar que sin caché ni DNS real la consulta falla, validando que la respuesta no aparece por error.

Resultados experimentales y análisis

Las pruebas de conectividad confirmaron que h1, h2, h3 y h4 alcanzan al servidor DNS 10.0.1.100 por medio del switch P4. Luego se validaron los escenarios HIT, MISS y WATCH. En HIT inicial, el servidor DNS no necesita estar activo, porque las entradas fueron precargadas por el controlador. En MISS, se ejecutó primero una consulta sin servidor para confirmar timeout, y luego se levantó dns_server.py para validar la respuesta real. En WATCH, el caché inició vacío, el updater aprendió tres dominios desde respuestas del servidor y después las consultas respondieron desde el switch. Un ejemplo de los resultados de las pruebas de latencia realizadas se puede observar en la Tabla 4.

Tabla 4.

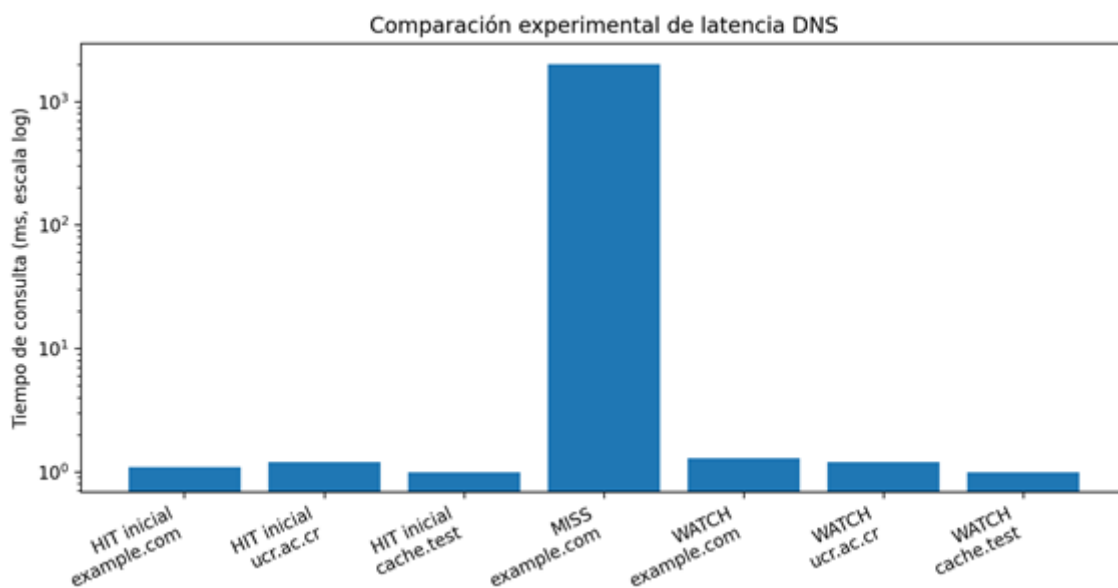
Resultados de latencia observados en las pruebas automáticas.

Escenario	Dominio	Media	Mín	Máx	N
HIT inicial	example.com	1.1 ms	0 ms	2 ms	10
HIT inicial	ucr.ac.cr	1.2 ms	1 ms	2 ms	10
HIT inicial	cache.test	1.0 ms	0 ms	2 ms	10
MISS con servidor	example.com	2029.6 ms	2020 ms	2033 ms	10
WATCH/HIT aprendido	example.com	1.3 ms	1 ms	2 ms	10
WATCH/HIT aprendido	ucr.ac.cr	1.2 ms	1 ms	2 ms	10

WATCH/HIT aprendido	cache.test	1.0 ms	0 ms	2 ms	10
------------------------	------------	--------	------	------	----

Figura 3.

Comparación de latencia entre HIT, MISS y WATCH. Se usa escala logarítmica por la diferencia entre milisegundos y segundos.



En la Figura 3 se puede observar que el resultado más importante es la diferencia entre HIT y MISS. Las respuestas desde caché P4 se mantienen entre 1 y 3 ms en las corridas observadas. En cambio, el MISS con servidor Scapy ronda los 2 s. Esta diferencia no significa que DNS real siempre tarde 2 s; más bien refleja que el servidor simulado usa Scapy en espacio de usuario, y que dig necesitó esperar una respuesta mucho más lenta que una respuesta construida directamente en el switch.

El watch-test confirma que la actualización dinámica funciona. Inicialmente, example.com no resuelve porque el caché está vacío y el servidor aún no está levantado. Después, el servidor responde los tres dominios y el updater imprime tres actualizaciones: example.com en índice 97, ucr.ac.cr en índice 882 y cache.test en índice 355. La segunda ronda de consultas responde con tiempos de HIT, lo que valida la integración entre servidor, updater, registers y pipeline P4.

En términos de análisis de tráfico, el comportamiento esperado es claro: en HIT no se debe observar una nueva consulta hacia el servidor DNS, mientras que en MISS sí se observa la consulta y la respuesta sobre UDP/53. Esto permite demostrar que la caché no es solamente una optimización de aplicación, sino una función realmente ejecutada en el plano de datos del switch.

Limitaciones y trabajo futuro

La implementación cumple el objetivo principal: demostrar una caché DNS mínima en P4 con soporte para consultas tipo A, almacenamiento en registers, lógica HIT/MISS, contador de eventos y actualización dinámica desde el control plane. Los resultados muestran que las consultas cacheadas se resuelven con latencia muy baja en el entorno BMv2, mientras que los MISS dependen del servidor DNS simulado.

La principal limitación es el parsing de nombres DNS. DNS usa nombres de longitud variable, mientras que P4/BMv2 no permite procesar bucles arbitrarios dentro del parser como lo haría un programa general en Python o C. Por esta razón, el prototipo reconoce dominios controlados y usa una clave parcial. Esta decisión es razonable para una demostración académica, pero una versión de producción requeriría una estrategia más robusta para manejar nombres de longitud variable, colisiones y diferentes tipos de registro.

Otra limitación es la política de reemplazo. El caché actual no implementa TTL real de DNS ni expulsión automática de entradas. Tampoco valida firmas DNSSEC ni maneja respuestas negativas. En escenarios reales, estas funciones son importantes para evitar respuestas obsoletas o inconsistentes. Por eso, el sistema debe entenderse como prueba de concepto de in-network caching, no como sustituto completo de un resolver DNS.

Como trabajo futuro, se propone agregar TTL por entrada, política de reemplazo basada en frecuencia, soporte para más dominios, un hash más robusto,

uso de Count-Min Sketch para detectar dominios populares y exportación automática de métricas. También sería útil evaluar el prototipo en hardware P4 real o en una plataforma con mayor fidelidad temporal que BMv2. Finalmente, se podrían capturar PCAPs comparando HIT y MISS para fortalecer la evidencia experimental del informe.

Conclusiones

Con este proyecto se logró implementar una caché DNS mínima en el plano de datos usando P4, BMv2 y Mininet. La solución permitió que el switch programable analizara consultas DNS tipo A, revisara entradas almacenadas en registers y respondiera directamente cuando existía una coincidencia. Esto demuestra que P4 puede usarse no solo para forwarding tradicional, sino también para mover funciones simples de aplicación hacia la red.

Las pruebas realizadas confirmaron el funcionamiento de los tres escenarios principales. En el caso HIT, las consultas fueron respondidas desde el switch sin necesidad de levantar el servidor DNS. En el caso MISS, la consulta fue reenviada al servidor DNS simulado y se obtuvo una respuesta correcta. Finalmente, en watch-test se comprobó que el controlador pudo aprender respuestas DNS reales y escribirlas dinámicamente en los registers del switch, permitiendo que consultas posteriores fueran respondidas desde caché.

Los resultados de latencia muestran una diferencia clara entre responder desde el switch y responder desde el servidor DNS simulado. Las respuestas desde caché se mantuvieron cerca de 1 a 3 ms, mientras que el MISS con servidor Scapy rondó los 2 segundos. Esta diferencia no significa que un servidor DNS real tenga siempre esa latencia, pero sí muestra que, dentro del entorno experimental, la respuesta construida directamente en el plano de datos fue mucho más rápida que la respuesta generada por el servidor en espacio de usuario.

También se identificaron limitaciones importantes. La principal es que DNS usa nombres de longitud variable, mientras que el parser de P4/BMv2 no está

pensado para procesar cadenas arbitrarias con la misma flexibilidad que un lenguaje general. Por eso, el prototipo trabaja con dominios controlados y una clave parcial. Además, la caché no implementa TTL real, política de reemplazo, DNSSEC ni manejo completo de distintos tipos de registros. Aun así, estas limitaciones son aceptables para una prueba de concepto enfocada en demostrar el procesamiento dentro del plano de datos.

En general, el proyecto cumplió su objetivo: mostrar de forma práctica cómo un switch programable puede responder consultas frecuentes usando información almacenada en el propio plano de datos. La implementación permite entender mejor la separación entre data plane y control plane, así como las ventajas y restricciones de usar P4 para funciones más allá del reenvío IP básico.

Referencias

- Cormode, G., & Muthukrishnan, S. (2005). An improved data stream summary: The Count-Min Sketch and its applications. *Journal of Algorithms*, 55(1), 58-75.
<https://doi.org/10.1016/j.jalgor.2003.12.001>
- Filsfils, C., Dukes, D., Previdi, S., Leddy, J., Matsushima, S., & Voyer, D. (2020). IPv6 Segment Routing Header (SRH) (RFC 8754). Internet Engineering Task Force.
<https://datatracker.ietf.org/doc/html/rfc8754>
- Hula, et al. (2020). Real-time DDoS mitigation in programmable data planes. *Proceedings of ACM SIGCOMM 2020*.
- Jin, X., Li, X., Zhang, H., Foster, N., Lee, J., Soulé, R., Kim, C., & Stoica, I. (2017). NetCache: Balancing key-value stores with fast in-network caching. *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP 2017)*, 121-136.
<https://doi.org/10.1145/3132747.3132764>
- Mahalingam, M., Dutt, D., Duda, K., Agarwal, P., Kreeger, L., Sridhar, T., Bursell, M., & Wright, C. (2014). Virtual eXtensible Local Area Network (VXLAN): A framework

for overlaying virtualized layer 2 networks over layer 3 networks (RFC 7348).

Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc7348>

P4 Language Consortium. (n.d.-a). P4 language specification. P4.org. <https://p4.org/p4-spec/>

P4 Language Consortium. (n.d.-b). P4 tutorials. GitHub. <https://github.com/p4lang/tutorials>